10-10-2025

Práctica 1: Expresiones Regulares

780018 - Procesadores del Lenguaje

Fuentes Lozano Álvaro
UNIVERSIDAD DE ALCALÁ

Contenido

1. Creación de las Expresiones Regulares 2
1.1. Identificador lenguaje programación ER 2
1.2. Cadenas alfabeto [a,b] con número par de aes 2
1.3. Números en coma flotante 3
1.4. Operaciones aritméticas de suma y resta entre variables y/o floats 3
1.4.1. Decisiones de diseño 3
1.4.2. ER 3
2. Transformación de ER a AFD simplificado 5
2.1.- Caso (a) Identificador lenguaje 5
2.2.- Caso (b) Cadenas alfabeto [a,b] con número par de aes 8
2.3.- Caso (c) Números en coma flotante 10
2.4.- Caso (d) Operaciones aritméticas de suma y resta entre variables y/o floats 13
3. Funcionamiento del programa 15
3.1. Arquitectura general 15
3.2. Flujo de ejecución 15
3.3. Casos Implementados 16
3.3.1. Caso (a) Identificadores 16
3.3.2. Caso (b) cadenas alfabeto [a,b] con número par de aes 16
3.3.3. Caso (c) Números en coma flotante 17
3.3.4. Caso (d) Operaciones aritméticas de suma y resta entre variables y/o floats ... 17

1. Creación de las Expresiones Regulares

1.1. Identificador lenguaje programación ER

Un identificador debe comenzar obligatoriamente por una letra y puede continuar con una secuencia de letras y/o números. Ejemplos válidos: var, var2, COUNTER.

Razonamiento paso a paso:

1. Primer carácter → letra.

Para asegurarnos de que siempre empiece por una letra, el primer símbolo debe pertenecer al conjunto de letras minúsculas o mayúsculas. En notación de expresiones regulares: [a-zA-Z].

2. Resto de caracteres → letras o dígitos.

A partir del segundo carácter, el identificador puede contener tanto letras como números, y además puede aparecer cero o más veces.

En notación de expresiones regulares: ([a-zA-Z0-9])*.

3. Combinación completa.

Uniendo ambas partes:

[a-zA-Z]([a-zA-Z0-9])*

1.2. Cadenas alfabeto [a,b] con número par de aes

Básicamente podemos tener cadenas formadas con esas dos letras pero que tengan un número par de aes, por lo que cadenas como baba serán válidas.

Razonamiento paso a paso:

1. Alfabeto restringido.

La cadena solo puede contener los símbolos a y b.

2. Condición principal: número par de a.

Para que haya un número par de a, estas deben aparecer siempre en pares:

- Puede no haber ninguna a (cero es par).
- Si aparece una a, necesariamente debe aparecer otra a más adelante.

3. Construcción de la expresión.

- Se permiten bloques de la forma: **b* a b* a b*;** que garantizan que las a aparecen en parejas, y entre ellas puede haber cualquier cantidad de b.
- Como puede haber varias parejas de a, repetimos ese bloque con el operador *.

- También hay que admitir cadenas que contengan únicamente b.

Por lo que la expresión regular será **b*(ab*ab*)*.**

1.3. Números en coma flotante

El objetivo es aceptar ejemplos como *25, 3.5, 33.02* y también negativos como *-25, -3.5 y -0.02*. Por decisiones de diseño no vamos a aceptar el símbolo + como indicador de positividad del número. Tampoco se admitirán números del estilo *3. o .5*

1. Lo primero es definir el conjunto de dígitos D = {0,1,2,3--,9}
2. Posteriormente tenemos en cuenta la parte entera (uno o más dígitos), donde un número tiene al menos un dígito D+. En notación de ER estándar esto se reconoce como [0-9]+
3. Después debemos tener en cuenta la parte fraccional, donde se debe ir si aparece el punto al menos seguido de un dígito: .[0-9]+. ¿Y lo hacemos opcional con (…)? → (.[0-9]+)?
4. Por último, añadimos la opcionalidad del signo – al inicio con -?

Siendo la combinación final teórica: **-?[0-9]+(.[0-9]+)?**

1.4. Operaciones aritméticas de suma y resta entre variables y/o floats

Debemos reconocer expresiones formadas por operandos (identificadores o números en coma flotante/enteros) encadenados por operadores **+** o **-**. Ejemplo válido: 3 - var + 2.5 + var2.

1.4.1. Decisiones de diseño

1. Operando = Identificador ∨ Número

Identificador (como en el (a)): empieza por letra y puede seguir con letras o dígitos.

Número (como en el (c)): entero o real con punto obligatorio si aparece.

2. Estructura de la expresión:

Un operando inicial seguido de cero o más repeticiones de: operador (+|-) + operando.

Esto evita expresiones como +3 o 3+ y también 3+-2 (no admitimos unario).

3. Espacios

El enunciado muestra espacios (3 - var + …). Permitimos espacios opcionales alrededor de los operadores. Para simplificar, consideramos solo el espacio normal (' '), aunque en código puedes pre-eliminar cualquier whitespace.

1.4.2. ER

1. El operando inicial puede ser un identificador o un número
2. Bloque repetible “operando + operando”

Tenemos a los operadores que puede ser un + o un – que los ponemos así (\+|\-) para no confundirlos con meta caracteres. Además, como en las decisiones de diseño hemos implementado los espacios opcionales alrededor del operador, tenemos esta expresión []* antes y después del operador.

Por último, tenemos el operando de cierre que puede ser o un IDF o un N.

3. Repetición del bloque 0 o más veces con *
4. Concatenación final

Quedaría tal que así: **(IDF | N) ([ ]* (+|-) [ ]* (IDF | N))***

Donde IDF es un identificador (a) y N es un número (c).

2. Transformación de ER a AFD simplificado

2.1.- Caso (a) Identificador lenguaje

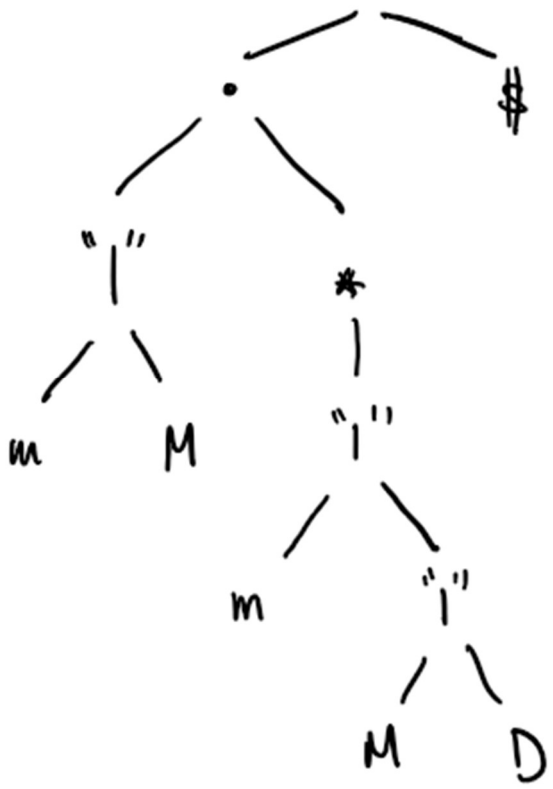
Partiendo de la ER: **[a-zA-Z]([a-zA-Z0-9])*,** vamos a hacer una simplificación para realizar el autómata finito no determinista.

He considerado como las siguientes letras los siguientes conjuntos:

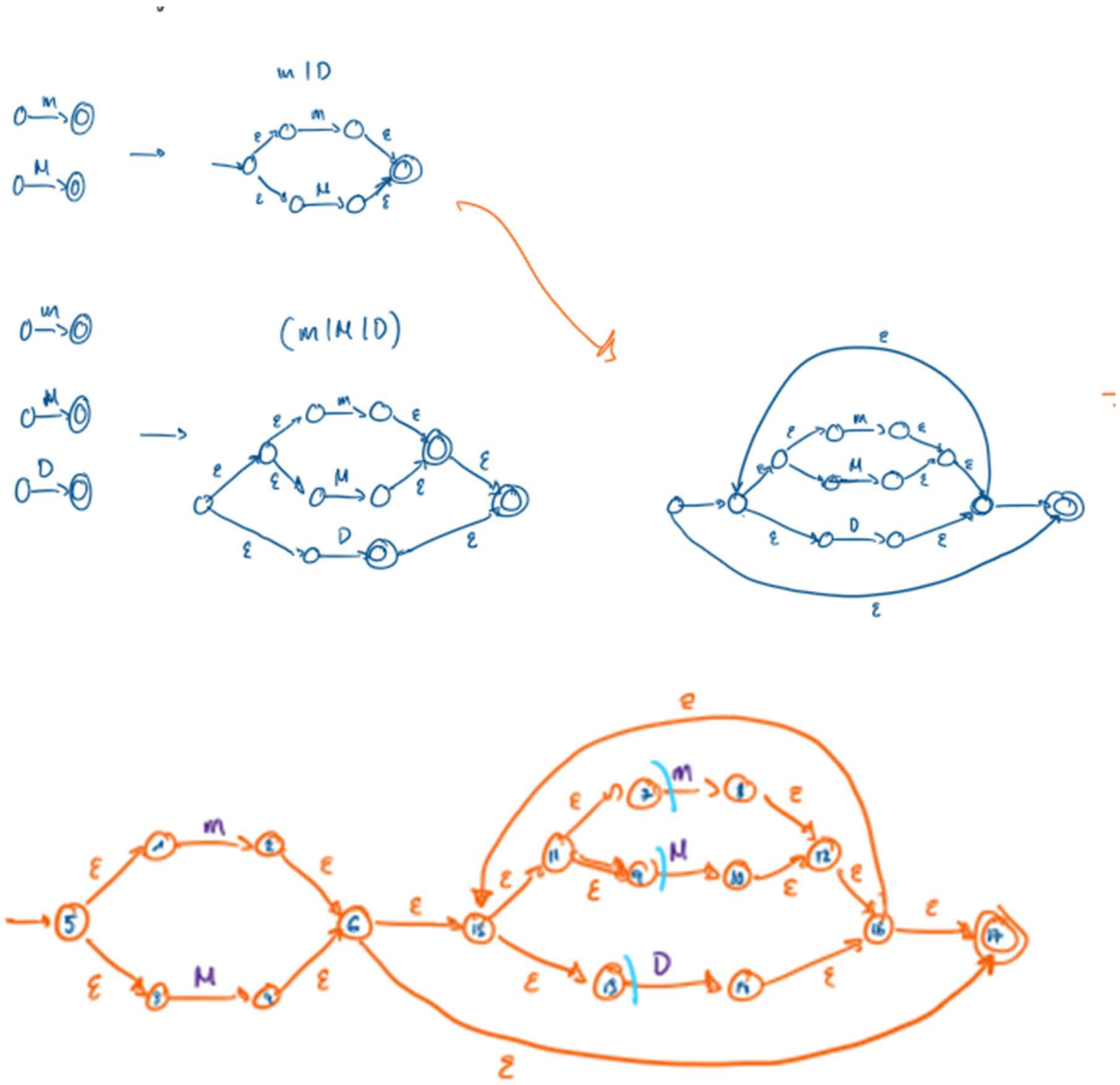
- m →[a-z]
- M →[A-Z]
- D → [0-9]

Partiendo de esta simplificación, la ER es la siguiente: **(m|M)(m|M|D)***

Lo primero que debemos hacer es construir el AFND para la anterior expresión, por lo que empezaremos creando el árbol del AFND: (realizado en mi ipad en one note)

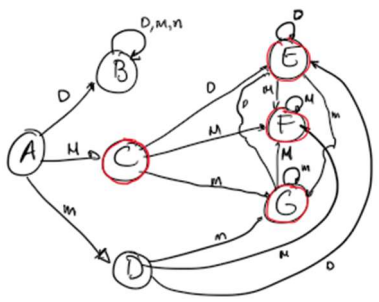


Partiendo del siguiente árbol, si vamos desde las hojas hasta los niveles más altos podemos sacar el siguiente AFND con un total de 17 estados.

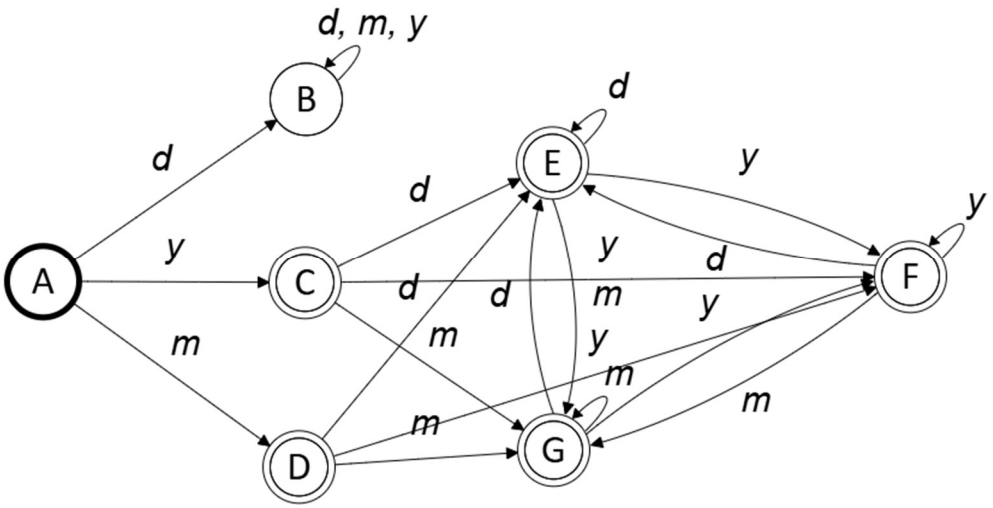


El siguiente paso sería construir el AFD a partir del AFND anterior. Empleando el conjunto de reglas vistas en el Tema 3 de clase he podido obtener el siguiente AFD.

Estados AFND	Estado del AFD	D	M	m
{5, 1, 2}	A	B	C	D
{ }	B	B	B	B
{4, 6, 7, 9, 11, 13, 15, 17}	C	E	F	G
{2, 6, 7, 9, 11, 13, 15, 17}	D	E	F	G
{7, 9, 11, 13, 14, 15, 16, 17}	E	E	F	G
{7, 9, 10, 11, 12, 13, 15, 16, 17}	F	E	F	G
{7, 8, 9, 91, 12, 13, 15, 16, 17}	G	E	F	G



El último paso sería simplificarlo. Podemos emplear dos métodos vistos en clase. El Teorema **Myhill-Nerodo** o por **equivalencia.**



La matriz de estados por consecuente será

Estados	m	M	D
A	(CDEFG)	(CDEFG)	B
(CDEFG)	(CDEFG)	(CDEFG)	(CDEFG)

Simplificación método equivalencia

$P_0 = \{A, B\}\{C, D, E, F, G\}$

$P_1 = \{A\}\{B\}\{C, D, E, F, G\}$

$P_2 = \{A\}\{B\}\{C, D, E, F, G\}$

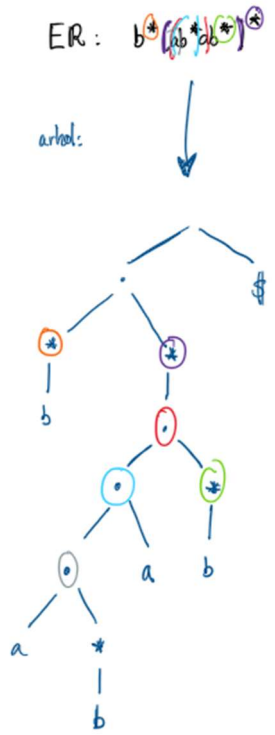
	d	M	m
A	B	D	C
B	B	B	B
C	E	F	G
D	E	F	G
E	E	F	G
F	E	F	G
G	E	F	G

autómata:

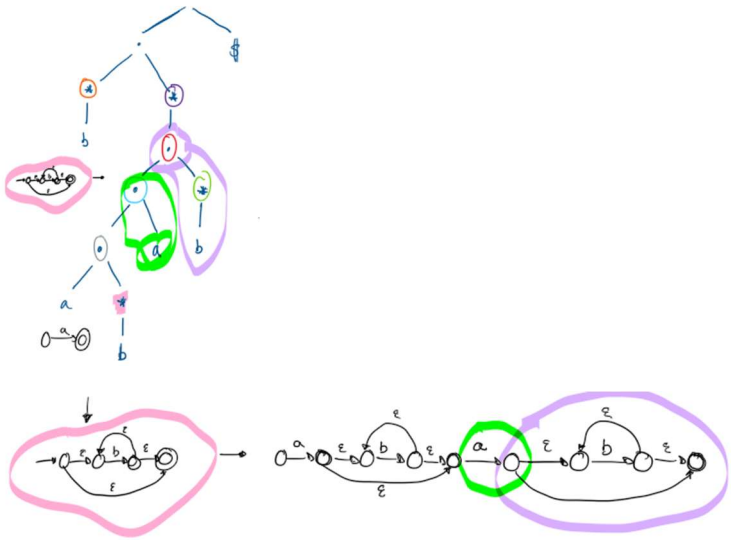
2.2.- Caso (b) Cadenas alfabeto [a,b] con número par de aes

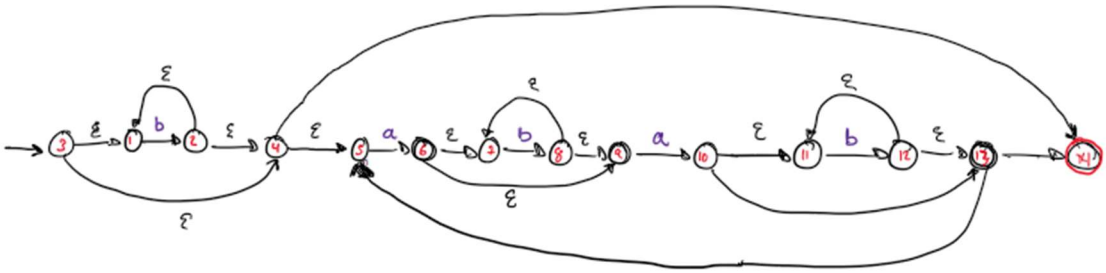
Partiendo de la expresión **b*(ab*ab*)*** vamos a realizar exactamente los mismos pasos que para el caso 2.1.

En este caso no he considerado realizar una conversión de tipos ya que el conjunto [a,b] es reducido.



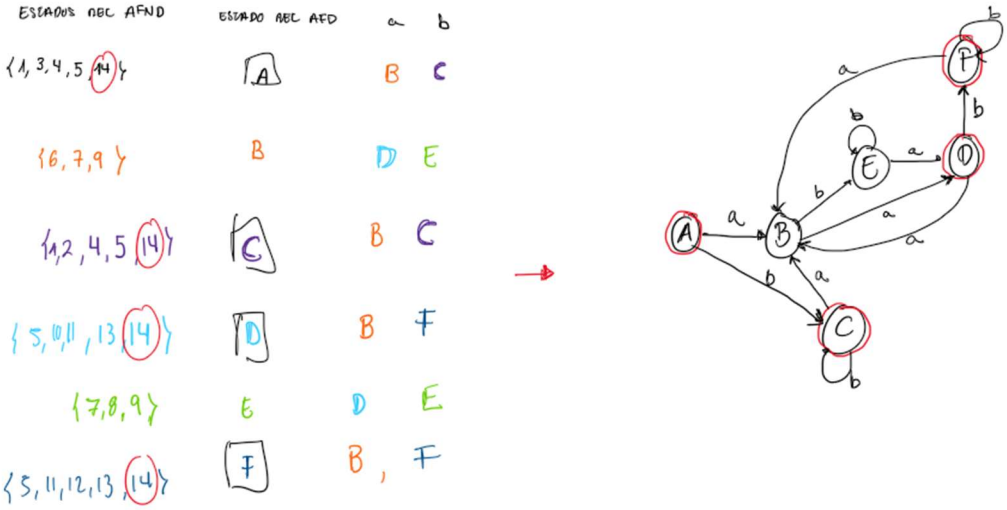
A partir del siguiente árbol podemos realizar siguiendo los pasos vistos en clase el AFND.



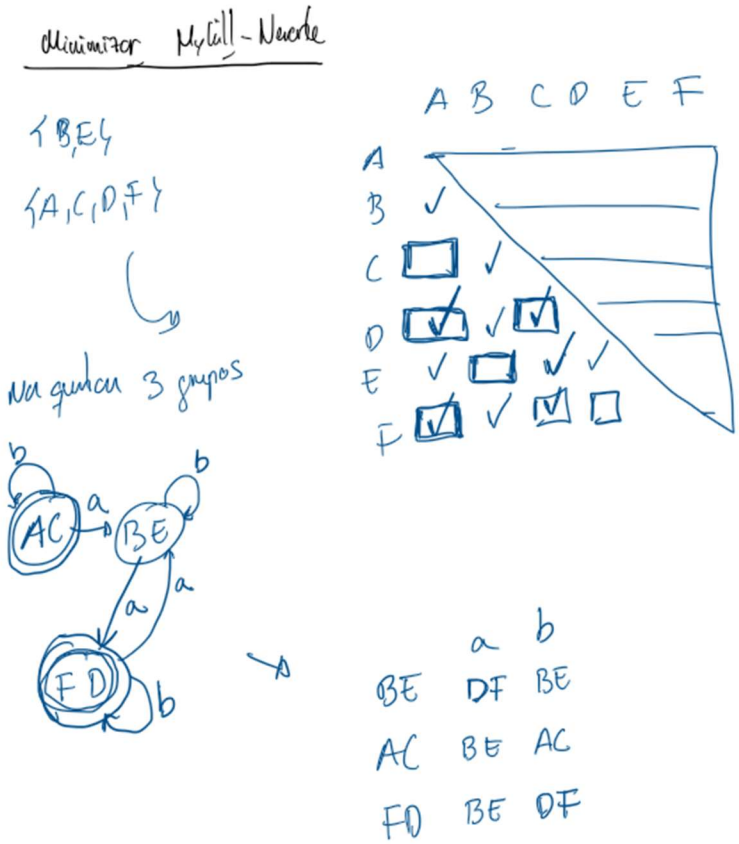


Con esto tendríamos que continuar con la transformación en un AFD.

Para transformarlo en AFD seguiremos la misma serie de iteraciones que para el caso 1 configurándose de esta manera.

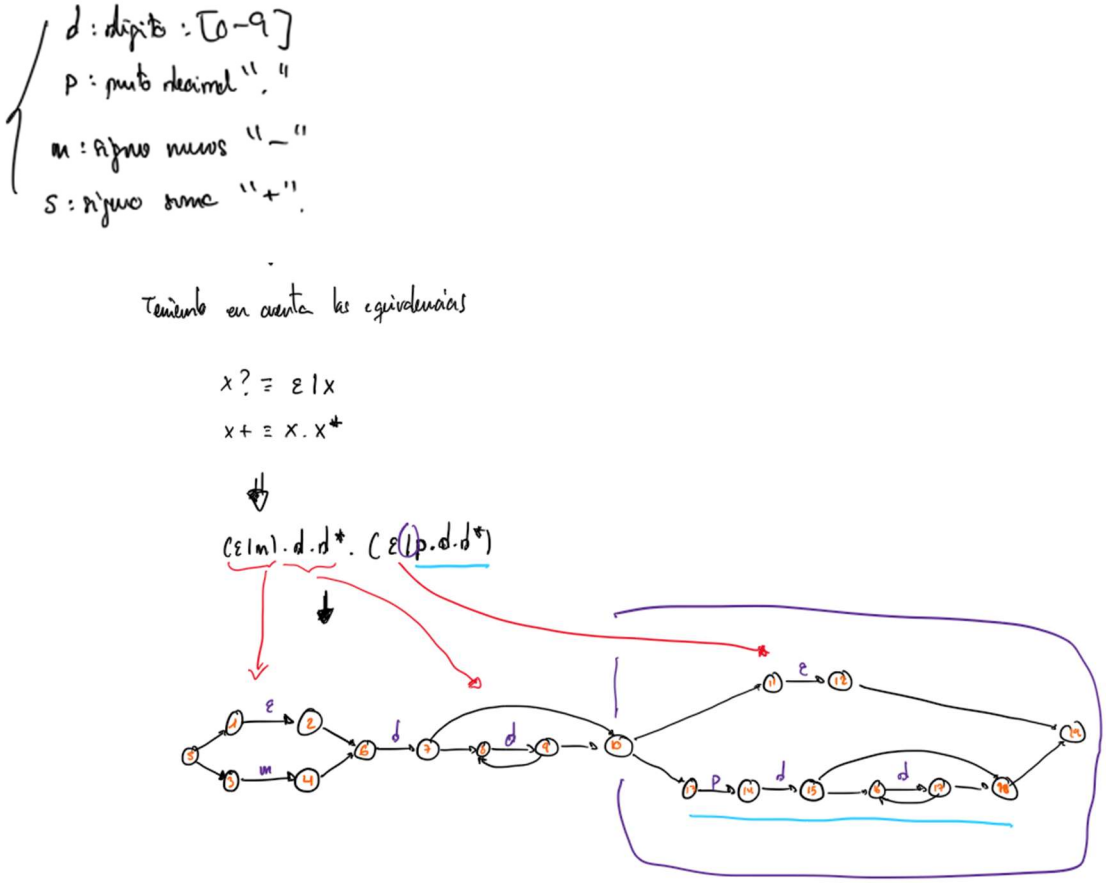


Finalmente quedaría minimizarlo. Esta vez lo hemos hecho mediante el otro teorema visto en clase dando como resultado el siguiente



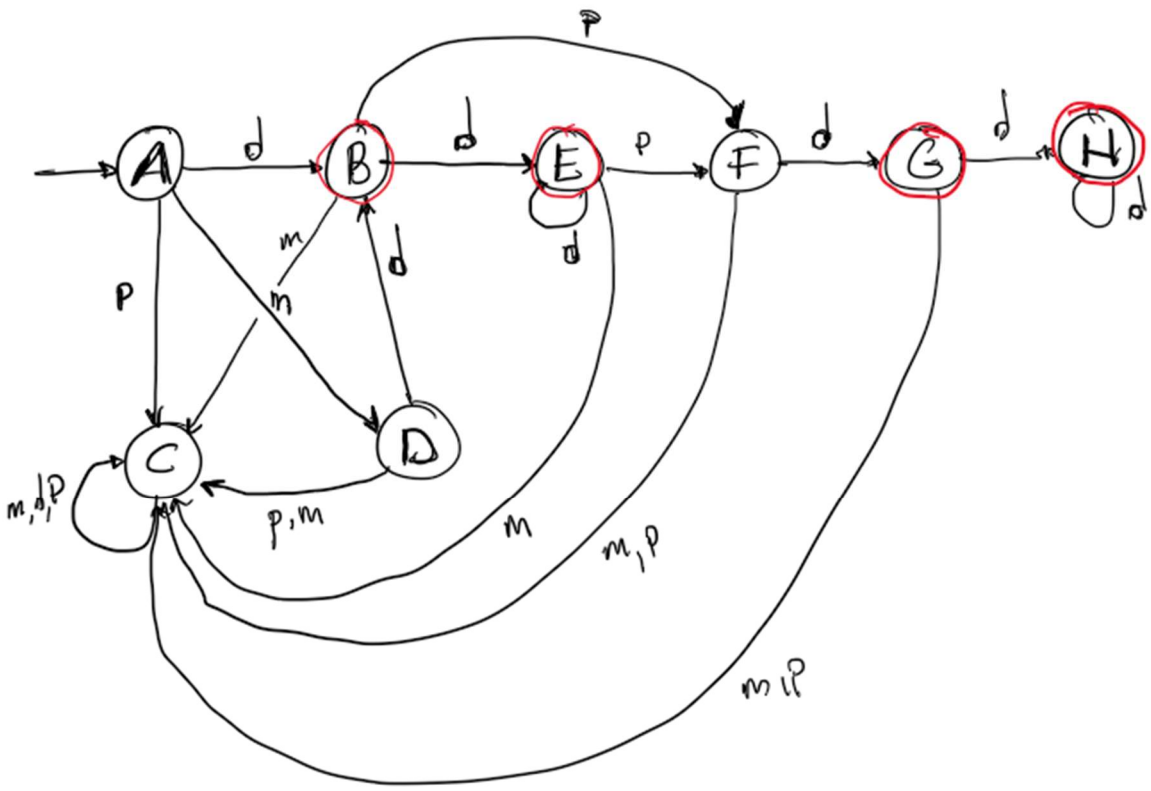
2.3.- Caso (c) Números en coma flotante

La expresión regular para este caso **-?[0-9]+(.[0-9]+)?**. Realizando las equivalencias y posterior desglose nos queda el siguiente AFND.

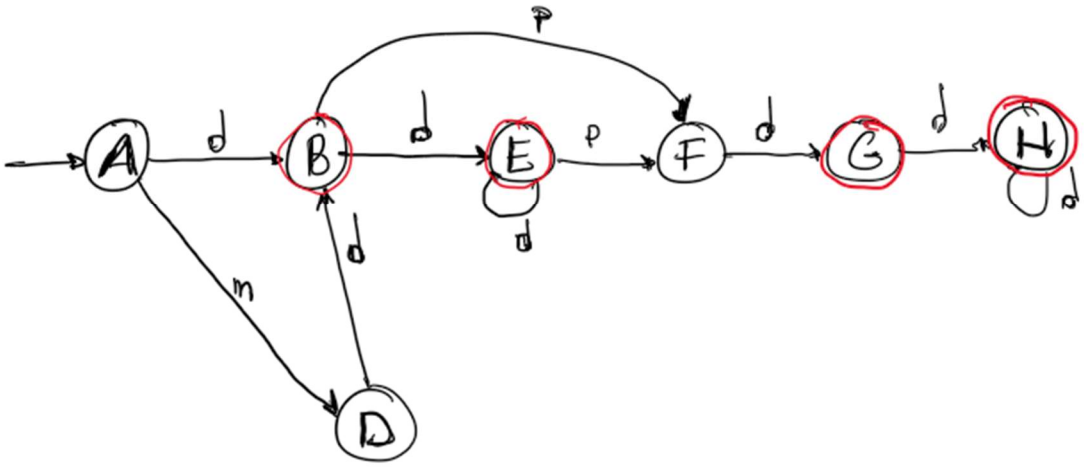


Posteriormente realizamos el AFD resultante del anterior AFND quedando el siguiente resultado.

Estado del AFND	Estado del AFD	d	p	m
{1,2,3,5,6}	A	B	C	D
{7,8,10,11,12,13,14,19}	B	E	F	C
{ }	C	C	C	C
{4,6}	D	B	C	C
{8,9,10,11,12,13,14,19}	E	E	F	C
{14}	F	G	C	C
{15,16,18,19}	G	H	C	C
{16,17,18,19}	H	H	C	C



Para simplificar el AFD, dado que hay un estado de error **C** lo vamos a suprimir.



Finalmente quedaría minimizar el AFD. Que bajo el método de equivalencia quedaría el siguiente:

aut. equivalencia

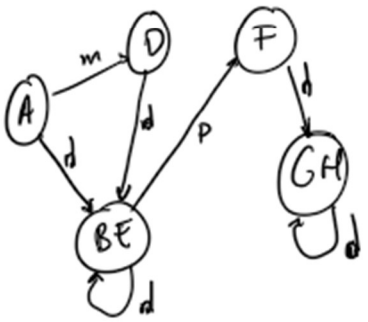
$P_0 = \{A, \quad, D, F\} \{B, E, G, H\}$

$P_1 = \{A\} \qquad \{D, F\} \{B, E, G, H\}$

$P_2 = \{A\} \{D, F\} \{B, E\} \quad \{G, H\}$

$P_3 = \{A\} \{D, F\} \{B, E\} \{G, H\}$

$P_4 = \{A\} \{D\} \{F\} \{B, E\} \{G, H\}$



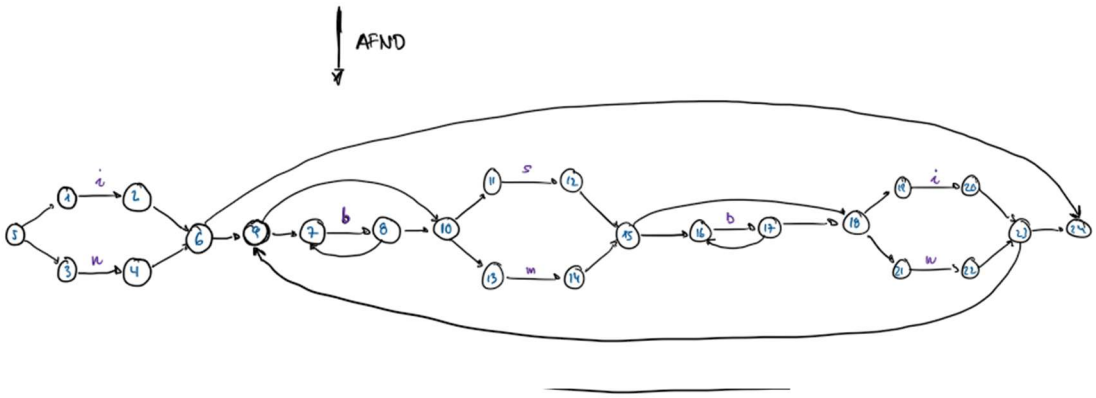
Estados	p	d	m
A	—	BE	D
BE	F	BE	—
D	—	BE	—
F	—	GH	—
GH	—	GH	—

El AFD simplificado resultante es el configurado a partir de la tabla de estados superior.

2.4.- Caso (d) Operaciones aritméticas de suma y resta entre variables y/o floats

El último caso es el basado en la expresión regular **(IDF | N) ([ ]* (+|-) [ ]* (IDF | N))***, donde IDF es un identificador, N un número real, [] un espacio en blanco y + y – los respectivos símbolos de suma y resta.

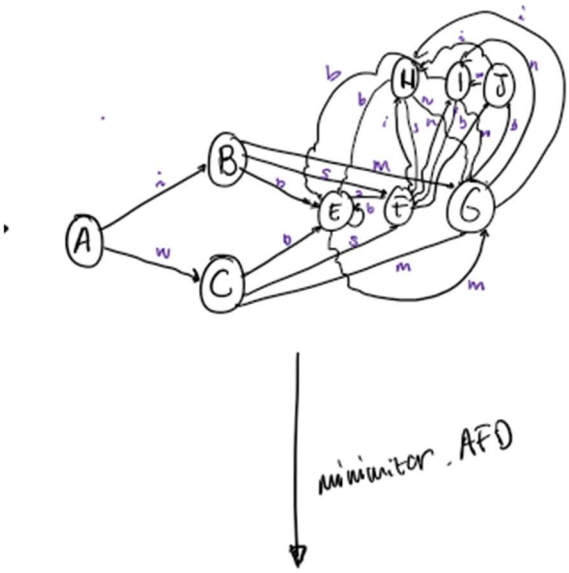
El procedimiento seguido es el mismo que para los casos anteriores, donde a partir de un árbol de decisión construimos el AFND.



AFD

Estados AFND	Estados AFD	i	n	b	s	m
{1,3,5}	A	B	C	D	D	D
{2,6,7,9,10,11,13,24}	B	D	D	E	F	G
{4,6,7,9,10,11,13,24}	C	D	D	E	F	G
{ }	D	D	D	D	D	D
{7,8,10,11,13	E	D	D	E	F	G
{12,15,16,18,19,21}	F	H	I	J	D	D
{14,15,16,18,19,21}	G	H	I	J	D	D
{7,9,10,11,13,20,23,24}	H	D	D	E	F	G
{7,9,10,11,13,22,23,24}	I	D	D	E	F	G
{16,17,18,19,21}	J	H	I	D	D	D

Finalmente solo quedaría minimizar el AFD. Lo realice con el teorema de equivalencia.

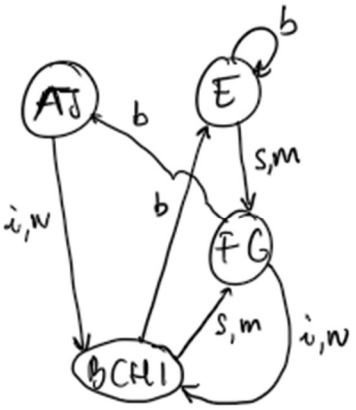


minimizar AFD

$P_0 = \{A, E, F, G, J\} \{B, C, H, I\}$

$P_1 = \{AJ\} \{E\} \{F, G\} \{B, C, H, I\}$

$P_2 = \{AJ\} \{E\} \{F, G\} \{B, C, H, I\}$



	b	i	m	w	s
E	E	–	FG	E	FG
FG	AJ BCHI	–	BCHI	–	
(BCHI)	E	–	FG	–	FG
AJ	–	BCHI	–	BCHI	–

3. Funcionamiento del programa

En esta sección describiré la arquitectura y el flujo de ejecución del programa desarrollado para la PL1. El objetivo es ejecutar, sobre una matriz de transición de estados (AFD minimizado), una máquina de estados genérica que acepte o rechace cadenas de entrada de acuerdo con la ER seleccionada en cada caso (a–d).

3.1. Arquitectura general

El núcleo es una clase de autómata determinista (DFA) parametrizada por:

- Nombres de estados.
- Alfabeto de símbolos.
- Matriz de transición ($|Q|\times|\Sigma|$) con -1 para rechazos inmediatos (ya que en el AFD minimizado el estado de error; si lo había, tal cual lo vimos en clase lo he suprimido).
- Estado inicial y conjunto de estados de aceptación.

Sobre el DFA se aplica un mapeador de caracteres que transforma cada carácter de la entrada en un índice del alfabeto (p. ej., ‘m’=minúscula, ‘M’=mayúscula, ‘d’=dígito, o símbolos ‘+’, ‘−’, ‘ ’, etc.). Este mapeo se define mediante reglas (predicados) y se reutiliza entre casos.

Implementación: la clase DFA expone los métodos reset() para inicializar, step(estado, símbolo) para transición y run(mapper, texto) para ejecutar sobre una cadena, devolviendo (acepta, traza).

La traza almacena la secuencia (estado, símbolo, siguiente_estado) para poder explicar el recorrido y depurar.

El mapeador se construye con mapper_from_rules(alfabeto, reglas), asociando cada predicado a su símbolo. Si ningún predicado se cumple para un carácter, se devuelve -1 y la ejecución se rechaza.

3.2. Flujo de ejecución

1) Selección del caso (a/b/c/d). El programa carga la matriz, alfabeto y estados de aceptación correspondientes.
2) Entrada de la cadena a analizar.
3) Para cada carácter, el mapeador lo convierte en un símbolo del alfabeto; el DFA aplica la transición.
4) Si alguna transición no existe (-1), la cadena se rechaza.
5) Tras consumir toda la entrada, la cadena se acepta si el estado actual es de aceptación.

A continuación voy a mostrar el funcionamiento del programa en Python para cada caso (a), (b), (c), (d), con capturas de ejemplos aceptados y rechazados según las ER y AFD’s planteadas y las reglas o decisiones de diseño que he tomado en algún caso en específico.

3.3. Casos Implementados

3.3.1. Caso (a) Identificadores

La expresión regular de este caso es **[a-zA-Z]([a-zA-Z0-9])*** que traducido queda como **(m|M)(m|M|D)*.**

Se acepta si comienza por mayúscula o minúscula y puede continuar con letras o dígitos.

```
PS C:\Users\Álvaro FL> & "C:/Users/Álvaro FL/AppData/Local/Programs/Python/Python312/python.exe" "c:/Users/Álvaro FL/Downloads/pl1_maquina.py"
PL1 – Máquina de estados genérica (DFA)
Casos disponibles: a, b, c, d
Elige caso (demo/a/b/c/d): a
Estados: ['A', 'CDEFG'] | Inicio: A | Aceptación: ['CDEFG']
Introduce cadenas (ENTER vacío para salir):
> a23
A -[m]-> CDEFG -> CDEFG -[D]-> CDEFG -> CDEFG -[D]-> CDEFG
Resultado: ACEPTADA ✅

> uyuyuuy
A -[m]-> CDEFG -> CDEFG -[m]-> CDEFG -> CDEFG -[m]-> CDEFG -> CDEFG -[m]-> CDEFG -> CDEFG -[m]-> CDEFG -> CDEFG -[m]-> CDEFG -> CDEFG -[m]-> CDEFG
Resultado: ACEPTADA ✅

> Mal90
A -[M]-> CDEFG -> CDEFG -[m]-> CDEFG -> CDEFG -[m]-> CDEFG -> CDEFG -[D]-> CDEFG -> CDEFG -[D]-> CDEFG
Resultado: ACEPTADA ✅

> 9Meyideo
A -[D]-> X
Resultado: RECHAZADA ❌

> 9898
A -[D]-> X
Resultado: RECHAZADA ❌

> eded efef
A -[m]-> CDEFG -> CDEFG -[m]-> CDEFG -> CDEFG -[m]-> CDEFG -> CDEFG -[m]-> CDEFG -> CDEFG -[<?>]-> X
Resultado: RECHAZADA ❌

> a 34
A -[m]-> CDEFG -> CDEFG -[<?>]-> X
Resultado: RECHAZADA ❌

>
```

Se aceptarán todos los casos descritos previamente y se rechazarán casos como el identificador que empieza por un número, solo dígitos, palabras o combinaciones espaciadas.

Como se puede ver, en el programa quedan descritos los estados posibles resultantes de mi AFD, el estado de inicio y lo(s) de aceptación, viéndose la trazabilidad para cada cadena de caracteres introducida.

3.3.2. Caso (b) cadenas alfabeto [a,b] con número par de aes

La expresión regular de este caso es **b*(ab*ab*)*** donde solo se aceptarán cadenas formadas con esos dos caracteres y que tengan un número par de aes.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                                                                    Python

PS C:\Users\Álvaro FL> & "C:/Users/Álvaro FL/AppData/Local/Programs/Python/Python312/python.exe" "c:/Users/Álvaro FL/Downloads/pl1_maquina.py"
PL1 – Máquina de estados genérica (DFA)
Casos disponibles: a, b, c, d
Elige caso (demo/a/b/c/d): b
Estados: ['AC', 'BE', 'DF'] | Inicio: AC | Aceptación: ['AC', 'DF']
Introduce cadenas (ENTER vacío para salir):
> abab
AC -[a]-> BE -> BE -[b]-> BE -> BE -[a]-> DF -> DF -[b]-> DF
Resultado: ACEPTADA ✅

> abaaba
AC -[a]-> BE -> BE -[b]-> BE -> BE -[a]-> DF -> DF -[a]-> BE -> BE -[b]-> BE -> BE -[a]-> DF
Resultado: ACEPTADA ✅

> abhbhbsa
AC -[a]-> BE -> BE -[b]-> BE -> BE -[<?>]-> X
Resultado: RECHAZADA ❌

> abbababababbababab
AC -[a]-> BE -> BE -[b]-> BE -> BE -[b]-> BE -> BE -[a]-> DF -> DF -[b]-> DF -> DF -[a]-> BE -> BE -[b]-> BE -> BE -[a]-> DF -> DF -[b]-> DF -> DF -[a]-> BE -> BE -[b]-> BE -> BE -[a]-> DF -> DF -[b]-> DF -> DF -[a]-> BE -> BE -[b]-> BE -> BE -[a]-> DF -> D
F -[b]-> DF
Resultado: ACEPTADA ✅

> abaaaaaa
AC -[a]-> BE -> BE -[b]-> BE -> BE -[a]-> DF -> DF -[a]-> BE -> BE -[a]-> DF -> DF -[a]-> BE -> BE -[a]-> DF -> DF -[a]-> BE
Resultado: RECHAZADA ❌

> ab ab
AC -[a]-> BE -> BE -[b]-> BE -> BE -[<?>]-> X
Resultado: RECHAZADA ❌

>
```

3.3.3. Caso (c) Números en coma flotante

La expresión regular empleada para este caso fue **-?[0-9]+(.[0-9]+)?** Que traducida queda como **(ε|m)d+(ε|pd+).**

Se aceptan cadenas como 3.15, 67, 0.8922, pero nunca se aceptarán cadenas del estilo ".09" o "3.", ni mucho menos cualquiera que no sea un número. Se aceptarán también número negativos con la forma -3,22323, pero los números positivos no necesitan el +3434 y de hecho no será aceptado por normas de diseño, ya que he considerado que los positivos son los que no tienen signo y los negativos los que tienen el – delante.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Álvaro FL> & "C:/Users/Álvaro FL/AppData/Local/Programs/Python/Python312/python.exe" "c:/Users/Álvaro FL/Downloads/pl1_maquina.py"
PL1 – Máquina de estados genérica (DFA)
Casos disponibles: a, b, c, d
Elige caso (demo/a/b/c/d): c
Estados: ['A', 'BE', 'D', 'F', 'GH'] | Inicio: A | Aceptación: ['BE', 'GH']
Introduce cadenas (ENTER vacío para salir):
> 3.15
A -[d]-> BE -> BE -[p]-> F -> F -[d]-> GH -> GH -[d]-> GH
Resultado: ACEPTADA ✅

> -2.3
A -[m]-> D -> D -[d]-> BE -> BE -[p]-> F -> F -[d]-> GH
Resultado: ACEPTADA ✅

> 2323
A -[d]-> BE -> BE -[d]-> BE -> BE -[d]-> BE -> BE -[d]-> BE
Resultado: ACEPTADA ✅

> 0.
A -[d]-> BE -> BE -[p]-> F
Resultado: RECHAZADA ❌

> .3434
A -[p]-> X
Resultado: RECHAZADA ❌

> asd3
A -[<?>]-> X
Resultado: RECHAZADA ❌

>
```

3.3.4. Caso (d) Operaciones aritméticas de suma y resta entre variables y/o floats

Los operandos válidos son el identificador de (a) y el número de (c) y se reutilizan ambos mapeadores. Además, por decisiones de diseño he permitido que entre el operando1 y el símbolo + o -, y viceversa, puedan existir espacios en blanco por si se escapa.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                                                    Python

PS C:\Users\Álvaro FL> & "C:/Users/Álvaro FL/AppData/Local/Programs/Python/Python312/python.exe" "c:/Users/Álvaro FL/Downloads/pl1_maquina.py"
Pl1 - Máquina de estados genérica (DFA)
Casos disponibles: a, b, c, d
Elige caso (demo/a/b/c/d): d
Estados: ['E', 'FG', 'BCHI', 'AJ'] | Inicio: AJ | Aceptación: ['BCHI']
Introduce cadenas (ENTER vacío para salir):
> 3 + 3
Tokens: nbsbn
AJ -[n]-> BCHI -> BCHI -[b]-> E -> E -[s]-> FG -> FG -[b]-> AJ -> AJ -[n]-> BCHI
Resultado: ACEPTADA ✅

> 3 + denien
Tokens: nbsbi
AJ -[n]-> BCHI -> BCHI -[b]-> E -> E -[s]-> FG -> FG -[b]-> AJ -> AJ -[i]-> BCHI
Resultado: ACEPTADA ✅

> 989 + 3jdd
Tokens: nbsbni
AJ -[n]-> BCHI -> BCHI -[b]-> E -> E -[s]-> FG -> FG -[b]-> AJ -> AJ -[n]-> BCHI -> BCHI -[i]-> X
Resultado: RECHAZADA ❌

> 334 - dd3
Tokens: nbmbi
AJ -[n]-> BCHI -> BCHI -[b]-> E -> E -[m]-> FG -> FG -[b]-> AJ -> AJ -[i]-> BCHI
Resultado: ACEPTADA ✅

> 3434   + 33434
Tokens: nbsbn
AJ -[n]-> BCHI -> BCHI -[b]-> E -> E -[s]-> FG -> FG -[b]-> AJ -> AJ -[n]-> BCHI
Resultado: ACEPTADA ✅

> mifidf -deded
Tokens: ibmi
AJ -[i]-> BCHI -> BCHI -[b]-> E -> E -[m]-> FG -> FG -[i]-> BCHI
Resultado: ACEPTADA ✅

> mfirfrf +frfr
Tokens: ibsi
AJ -[i]-> BCHI -> BCHI -[b]-> E -> E -[s]-> FG -> FG -[i]-> BCHI
Resultado: ACEPTADA ✅

> 34434 +frefr
Tokens: nbsi
AJ -[n]-> BCHI -> BCHI -[b]-> E -> E -[s]-> FG -> FG -[i]-> BCHI
Resultado: ACEPTADA ✅

> 45345 +3434
Tokens: nbsn
AJ -[n]-> BCHI -> BCHI -[b]-> E -> E -[s]-> FG -> FG -[n]-> BCHI
Resultado: ACEPTADA ✅

> dnedenue +e dede+d e
Tokens: ibsibisibi
AJ -[i]-> BCHI -> BCHI -[b]-> E -> E -[s]-> FG -> FG -[i]-> BCHI -> BCHI -[b]-> E -> E -[i]-> X
Resultado: RECHAZADA ❌
```

.